

Data Structures Lab File

Peerzada Mohammad Sameem Makhdoomi
2124CUKMR09
3rd Semester

Table of contents

Sno.	Programs	Page no.
1	Creating, adding and counting the nodes in Linkedlist	5
2	Basic operations like insert, delete, search on linkedlist	6
3	PUSH, POP operations of stack using arrays	9
4	PUSH, POP operations of stack using pointers.	11
5	Add, delete operations of a queue using arrays	13
6	Add, delete operations of a queue using pointers	14
7	Infix to postfix using a stack	15
8	Circular Linked list demonstration	18
9	Insert element at beginning in Circular Singly Linkedlist	19

10	Insert element at end in Circular Singly Linkedlist	20
11	Insert element at a particular position in Circular Singly Linkedlist	21
12	Creating a Circular Singly Linkedlist	22
13	Deleting first element of Singly Circular Linkedlist	23
14	Deleting Last element of Singly Circular Linkedlist	24
15	Insert element in Sorted Singly Linkedlist	25
16	Doubly Linkedlist demonstration	27
17	In-order, pre-order and post-order (binary tree traversals) using recursion	28
18	Binary Search	29
19	Insertion sort	30

20	Bubble Sort	31
21	Selection sort	32
22	Quick sort	33
23	Heap Sort	34
24	Merge sort	36

- Creating, adding and counting the nodes in Linkedlist

```c
#include<stdio.h>
#include<stdlib.h>

struct node{
        int data;
        struct node* link;
};

int main()
{
        int num, num1;

        struct node* head = (struct node*)malloc(sizeof(struct node));
        printf("Enter data for first node: \n");
        scanf("%d", &num);
        head->data = num;
        head->link = NULL;

        printf("Enter the number for 2nd node: \n");
        scanf("%d", &num1);
        struct node* current = (struct node*)malloc(sizeof(struct node));
        current->data = num1;
        current->link = NULL;

        head->link = current;

        printf("Linkedlist: ");

        struct node* temp = head;

        int count=0;

        do
        {
                count++;
                printf("%d ", temp->data);
                temp = temp->link;

        }while(temp != NULL);

        printf("\nNumber of nodes in linkedlist: %d", count);
        return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
Enter data for first node:
45
Enter the number for 2nd node:
77
Linkedlist: 45 77
Number of nodes in linkedlist: 2
```

- Program to perform basic operations like insert, delete, search on linked list.

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {  // define a node in the linked list
    int data;
    struct Node* next;
};
struct Node* newNode(int data) { // function to create a new node
    struct Node* node = (struct Node*) malloc(sizeof(struct Node));
    node->data = data;
    node->next = NULL;
    return node;
}
// function to insert a new node at the beginning of the linked list
void insertAtBeginning(struct Node** head, int data) {
    struct Node* node = newNode(data);
    node->next = *head;
    *head = node;
    printf("%d inserted successfully at the beginning.\n", data);
}
// function to insert a new node at the end of the linked list
void insertAtEnd(struct Node** head, int data) {
    struct Node* node = newNode(data);
    if (*head == NULL) {
        *head = node;
        printf("%d inserted successfully at the end.\n", data);
        return; }
    struct Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = node;
    printf("%d inserted successfully at the end.\n", data);
}
```

CONTINUED

```c
// function to delete a node with given data from the linked list
void deleteNode(struct Node** head, int data) {
    struct Node* temp = *head;
    struct Node* prev = NULL;
    while (temp != NULL && temp->data != data) {
        prev = temp;
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("%d not found in the linked list.\n", data);
        return;
    }
    if (prev == NULL) {
        *head = temp->next;
    } else {
        prev->next = temp->next;
    }
    free(temp);
    printf("%d deleted successfully from the linked list.\n", data);
}
// function to search for a node with given data in the linked list
void searchNode(struct Node* head, int data) {
    struct Node* temp = head;
    while (temp != NULL && temp->data != data) {
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("%d not found in the linked list.\n", data);
    } else {
        printf("%d found in the linked list.\n", data);
    }
}
```

CONTINUED

```c
// function to print the linked list
void printList(struct Node* head) {
    if (head == NULL) {
        printf("Linked list is empty.\n");
        return;
    }
    printf("Linked list: ");
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
// main function to test the linked list operations
int main() {
    struct Node* head = NULL;
    insertAtBeginning(&head, 3);
    insertAtBeginning(&head, 2);
    insertAtBeginning(&head, 1);
    insertAtEnd(&head, 4);
    insertAtEnd(&head, 5);
    printList(head);
    searchNode(head, 3);
    searchNode(head, 6);
    deleteNode(&head, 3);
    deleteNode(&head, 6);
    printList(head);
    return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
3 inserted successfully at the beginning.
2 inserted successfully at the beginning.
1 inserted successfully at the beginning.
4 inserted successfully at the end.
5 inserted successfully at the end.
Linked list: 1 2 3 4 5
3 found in the linked list.
6 not found in the linked list.
3 deleted successfully from the linked list.
6 not found in the linked list.
Linked list: 1 2 4 5
```

➢PUSH, POP operations of stack using arrays

```c
#include <stdio.h>
#include <stdlib.h>
// define a stack data structure
struct Stack {
    int top;
    int capacity;
    int* array;
};
// function to create a new stack
struct Stack* createStack(int capacity) {
    struct Stack* stack = (struct Stack*) malloc(sizeof(struct Stack));
    stack->top = -1;
    stack->capacity = capacity;
    stack->array = (int*) malloc(capacity * sizeof(int));
    return stack;
}
// function to check if the stack is full
int isFull(struct Stack* stack) {
    return stack->top == stack->capacity - 1;
}
// function to check if the stack is empty
int isEmpty(struct Stack* stack) {
    return stack->top == -1;
}
// function to push an element onto the stack
void push(struct Stack* stack, int data) {
    if (isFull(stack)) {
        printf("Stack overflow.\n");
        return;
    }
    stack->array[++stack->top] = data;
    printf("%d pushed onto the stack.\n", data);
}
```

CONTINUED

```c
int pop(struct Stack* stack) { //pop an element from the stack
    if (isEmpty(stack)) {
        printf("Stack underflow.\n");
        return -1; }
    int data = stack->array[stack->top--];
    printf("%d popped from the stack.\n", data);
    return data;
}
void printStack(struct Stack* stack) { // function to print the stack
    if (isEmpty(stack)) {
        printf("Stack is empty.\n");
        return;  }
    printf("Stack: ");
    for (int i = 0; i <= stack->top; i++) {
        printf("%d ", stack->array[i]); }
    printf("\n");
}
int main() { // main function to test the stack operations
    struct Stack* stack = createStack(5);
    push(stack, 1);
    push(stack, 2);
    push(stack, 3);
    push(stack, 4);
    push(stack, 5);
    push(stack, 6); // this will cause stack overflow
    printStack(stack);
    pop(stack);
    pop(stack);
    pop(stack);
    pop(stack);
    pop(stack);
    pop(stack); // this will cause stack underflow
    printStack(stack);
    return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
1 pushed onto the stack.
2 pushed onto the stack.
3 pushed onto the stack.
4 pushed onto the stack.
5 pushed onto the stack.
Stack overflow.
Stack: 1 2 3 4 5
5 popped from the stack.
4 popped from the stack.
3 popped from the stack.
2 popped from the stack.
1 popped from the stack.
Stack underflow.
Stack is empty.
```

➢PUSH,POP operations of stack using pointers

```c
#include <stdio.h>
#include <stdlib.h>
struct Node { // define a stack data structure
    int data;
    struct Node* next;
};
struct Node* newNode(int data) { // func to create a new node
    struct Node* node = (struct Node*) malloc(sizeof(struct Node));
    node->data = data;
    node->next = NULL;
    return node;
}
int isEmpty(struct Node* top) { // func to check if the stack is empty
    return top == NULL;
}
// function to push an element onto the stack
void push(struct Node** top, int data) {
    struct Node* node = newNode(data);
    node->next = *top;
    *top = node;
    printf("%d pushed onto the stack.\n", data);
}
// function to pop an element from the stack
int pop(struct Node** top) {
    if (isEmpty(*top)) {
        printf("Stack underflow.\n");
        return -1;
    }
    struct Node* temp = *top;
    *top = (*top)->next;
    int data = temp->data;
    free(temp);
    printf("%d popped from the stack.\n", data);
    return data;
}
```

CONTINUED

```c
// function to print the stack
void printStack(struct Node* top) {
    if (isEmpty(top)) {
        printf("Stack is empty.\n");
        return;
    }
    printf("Stack: ");
    while (top != NULL) {
        printf("%d ", top->data);
        top = top->next;
    }
    printf("\n");
}
// main function to test the stack operations
int main() {
    struct Node* top = NULL;
    push(&top, 1);
    push(&top, 2);
    push(&top, 3);
    push(&top, 4);
    push(&top, 5);
    push(&top, 6); // this will cause stack overflow
    printStack(top);
    pop(&top);
    pop(&top);
    pop(&top);
    pop(&top);
    pop(&top);
    pop(&top); // this will cause stack underflow
    printStack(top);
    return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
1 pushed onto the stack.
2 pushed onto the stack.
3 pushed onto the stack.
4 pushed onto the stack.
5 pushed onto the stack.
6 pushed onto the stack.
Stack: 6 5 4 3 2 1
6 popped from the stack.
5 popped from the stack.
4 popped from the stack.
3 popped from the stack.
2 popped from the stack.
1 popped from the stack.
Stack is empty.
```

➢Add, delete operations of a queue using arrays

```c
#include <stdio.h>
#define MAX_SIZE 100
int queue[MAX_SIZE], front = -1, rear = -1;
void enqueue(int element) { // func to add an element to the queue
    if (rear == MAX_SIZE - 1) {
        printf("Queue is full.\n");
    } else {
        if (front == -1) {
            front = 0;   }
        rear++;
        queue[rear] = element;
        printf("%d enqueued to queue.\n", element);}
}
void dequeue() { // func to remove an element from the queue
    if (front == -1 || front > rear) {
        printf("Queue is empty.\n");
    } else {
        int dequeuedElement = queue[front];
        front++;
        printf("%d dequeued from queue.\n", dequeuedElement); }
}
int main() {
    enqueue(1);
    enqueue(2);
    enqueue(3);
    enqueue(4);
    enqueue(5);
    dequeue();
    dequeue();
    dequeue();
    return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
1 enqueued to queue.
2 enqueued to queue.
3 enqueued to queue.
4 enqueued to queue.
5 enqueued to queue.
1 dequeued from queue.
2 dequeued from queue.
3 dequeued from queue.
```

➢Add, delete operations of a queue using pointers

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
struct Node* front = NULL;
struct Node* rear = NULL;
void enqueue(int element) { // func to add an element to the queue
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = element;
    newNode->next = NULL;
    if (front == NULL && rear == NULL) {
        front = rear = newNode;
        return; }
    rear->next = newNode;
    rear = newNode;
}
void dequeue() { // func to remove an element from the queue
    if (front == NULL) {
        printf("Queue is empty.\n");
        return;  }
    struct Node* temp = front;
    front = front->next;
    if (front == NULL) {
        rear = NULL;   }
    printf("%d dequeued from queue.\n", temp->data);
    free(temp);
}
```

```c
int main() {
    enqueue(1);
    enqueue(2);
    enqueue(3);
    enqueue(4);
    enqueue(5);
    dequeue();
    dequeue();
    dequeue();
    dequeue();
    dequeue();
    dequeue();
    return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
1 dequeued from queue.
2 dequeued from queue.
3 dequeued from queue.
4 dequeued from queue.
5 dequeued from queue.
Queue is empty.
```

- Infix to postfix using a stack

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAX_SIZE 100
struct Stack { // Stack structure
    int top;
    char arr[MAX_SIZE];
};
void initialize(struct Stack* stack){ // func to initialize stack
    stack->top = -1;
}
void push(struct Stack* stack, char element){ // push element onto stack
    if (stack->top == MAX_SIZE - 1) {
        printf("Stack Overflow\n");
        exit(1);
    }
    stack->top++;
    stack->arr[stack->top] = element;
}
char pop(struct Stack* stack) { // func to pop element from stack
    if (stack->top == -1) {
        printf("Stack Underflow\n");
        exit(1);
    }
    char element = stack->arr[stack->top];
    stack->top--;
    return element;
}
```

```c
// function to get the precedence of an operator
int precedence(char operator) {
    if (operator == '+' || operator == '-')
        return 1;
    else if (operator == '*' || operator == '/')
        return 2;
    else if (operator == '^')
        return 3;
    else
        return -1;
}
```

CONTINUED

```c
// function to convert infix expression to postfix expression
void infixToPostfix(char* infix, char* postfix) {
    struct Stack stack;
    initialize(&stack);
    int i = 0;
    int j = 0;
    while (infix[i] != '\0') {
        if (isalnum(infix[i])) {
 // if the character is an operand, add it to the postfix expression
            postfix[j] = infix[i];
            j++;
        } else if (infix[i] == '(') {
// if the character is a left parenthesis, push it onto the stack
            push(&stack, infix[i]);
        } else if (infix[i] == ')') {
// if the character is a right parenthesis, pop elements from the stack and
// add them to the postfix expression until a left parenthesis is encountered
            while (stack.top != -1 && stack.arr[stack.top] != '(') {
                postfix[j] = pop(&stack);
                j++;
            }
            if (stack.top == -1) {
                printf("Invalid Expression\n");
                exit(1);
            }
            pop(&stack); // pop the left parenthesis from the stack
        } else {
// if the character is an operator, pop operators from the stack and
// add them to the postfix expression until an operator  with lower precedence is encountered
            while (stack.top != -1 && precedence(infix[i]) <= precedence(stack.arr[stack.top])) {
                postfix[j] = pop(&stack);
                j++;
            }
            push(&stack, infix[i]); // push the operator onto the stack
        }
        i++;
    }
    while (stack.top != -1) {
// pop remaining operators from the stack and add them to the postfix expression
        postfix[j] = pop(&stack);
        j++;
    }
    postfix[j] = '\0'; // add null terminator to the postfix expression
}
```

CONTINUED

```c
int main() {
    char infix[MAX_SIZE];
    char postfix[MAX_SIZE];
    printf("Enter an infix expression: ");
    scanf("%s", infix);
    infixToPostfix(infix, postfix);
    printf("Postfix expression: %s\n", postfix);
    return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure/Run]
└─$ ./a.out
Enter an infix expression: a+b-c+d*e+f
Postfix expression: ab+c-de*+f+
```

➢Circular Linked list demonstration

```c
#include<stdio.h>
#include<stdlib.h>


struct node{
        int data;
        struct node* next;
};

struct node* circularSingly(int data)
{
        struct node* temp = malloc(sizeof(struct node));
        temp->data = data;
        temp->next = temp;
        return temp;
}

int main(){

        int data=34;
        struct node* tail;
        tail = circularSingly(data);

        printf("%d\n", tail->data);
        return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
34
```

- Insert element at beginning in Circular Singly Linkedlist

```c
#include<stdio.h>
#include<stdlib.h>

struct node{
        int data;
        struct node* next;
};

struct node* addToEmpty(int data)
{
        struct node* temp = malloc(sizeof(struct node));
        temp->data = data;
        temp->next = temp;
        return temp;
}

struct node* addAtBegin(struct node* tail, int data)
{
        struct node* newP  = malloc(sizeof(struct node));
        newP->data = data;
        newP->next = tail->next;
        tail->next = newP;
        return tail;
}

void print(struct node* tail)
{
         struct node* p = tail->next;
         do
         {
                 printf("%d ", p->data);
                 p = p->next;
         }while(p!=tail->next);
}

int main(){

         struct node* tail;
         printf("Adding a number: 10\n");
         tail = addToEmpty(10);
         printf("Adding at begin: 20\n");
         tail = addAtBegin(tail, 20);

         print(tail);
         return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
Adding a number: 10
Adding at begin: 20
20 10
```

➢Insert element at end in Circular Singly Linkedlist

```c
int main(){

        struct node* tail;
        printf("Adding a number: 10\n");
        tail = addToEmpty(10);
        printf("Add at begin: 20\n");
        tail = addAtBegin(tail, 20);

        printf("Add at end: 30\n");
        printf("Add at end: 40\n");
        tail = addAtEnd(tail,30);
        tail = addAtEnd(tail,40);

        printf("\nLinkedList: ");
        print(tail);
        return 0;
}
```

```c
struct node* addAtEnd(struct node* tail, int data)
{
        struct node* newP = malloc(sizeof(struct node));
        newP->data = data;
        newP->next = NULL;

        newP->next = tail->next;
        tail->next = newP;
        tail = tail->next;
        return tail;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ gcc insertion_circular_llist.c

┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
Adding a number: 10
Add at begin: 20
Add at end: 30
Add at end: 40

LinkedList: 20 10 30 40
```

- Insert element at a particular position in Circular Singly Linkedlist

```c
// add at a particular given position
struct node* addAfterPos(struct node* tail, int data, int pos)
{
        struct node* p = tail->next;
        struct node* newP = malloc(sizeof(struct node));
        newP->data = data;
        newP->next = NULL;

        while(pos > 1)
        {
                p = p->next;
                pos--;
        }
        newP->next = p->next;
        p->next = newP;
        if(p == tail)
        {
                tail = tail->next;
        }
        return tail;
}
```

```c
int main(){

        struct node* tail;
        printf("Adding a number: 10\n");
        tail = addToEmpty(10);
        printf("Add at begin: 20\n");
        tail = addAtBegin(tail, 20);
        printf("Add at end: 30\n");
        tail = addAtEnd(tail,30);

        printf("LinkedList: ");
        print(tail);

        printf("\n\nAdd an element 66 after 2nd position\n");
        tail = addAfterPos(tail, 66, 2);
        printf("LinkedList: ");
        print(tail);
        return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
Adding a number: 10
Add at begin: 20
Add at end: 30
LinkedList: 20 10 30

Add an element 66 after 2nd position
LinkedList: 20 10 66 30
```

➢Creating a Circular Singly Linkedlist

```
struct node* createList(struct node* tail)
{
        int i,n,data;
        printf("Enter the number of nodes of the linkedlist:\n");
        scanf("%d",&n);

        if(n == 0)
                return tail;
        printf("Enter the element 1: ");
        scanf("%d", &data);
        tail = addToEmpty(data);

        for(i=1; i<n; i++)
        {
                printf("Enter the element %d: ",i+1);
                scanf("%d", &data);
                tail = addAtEnd(tail, data);
        }
        return tail;
}

int main(){

        struct node* tail = NULL;
        tail = createList(tail);
        printf("LinkedList: ");
        print(tail);
        return 0;
}
```

```
(kali㉿ft10101)-[~/Documents/Study/Datastructure]
$ ./a.out
Enter the number of nodes of the linkedlist:
3
Enter the element 1: 11
Enter the element 2: 111
Enter the element 3: 1111
LinkedList: 11 111 1111
```

```
void print(struct node* tail)
{
        if(tail==NULL)
                printf("No node in the list ");
        else{
        struct node* p = tail->next;
        do
        {
                printf("%d ", p->data);
                p = p->next;
        } while(p!=tail->next);
        }
        printf("\n");
}
```

- Deleting first element of Singly Circular Linkedlist

```c
struct node* delFirst(struct node* tail)
{
        if(tail == NULL)
                return tail;
        if(tail->next == tail)
        {
                free(tail);
                tail = NULL;
                return tail;
        }
        struct node* temp = tail->next;
        tail->next = temp->next;
        free(temp);
        temp = NULL;
        return tail;
}

int main()
{
        struct node* tail = NULL;
        tail = createList(tail);
        printf("LinkedList before deletion: \n");
        print(tail);

        tail = delFirst(tail);
        printf("\nLinkedList after deletion: \n");
        print(tail);
        return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
Enter the number of nodes of the linkedlist:
4
Enter the element 1: 1
Enter the element 2: 11
Enter the element 3: 111
Enter the element 4: 1111
LinkedList before deletion:
1 11 111 1111

LinkedList after deletion:
11 111 1111
```

- Deleting Last element of Singly Circular Linkedlist

```
struct node* delLast(struct node* tail)
{
        if(tail == NULL)
                return tail;
        struct node* temp = tail->next;
        if(tail->next == tail)
        {
                free(tail);
                tail = NULL;
                return tail;
        }
        while(temp->next != tail)
        {
                temp = temp->next;
        }
        temp->next = tail->next;
        free(tail);
        tail = temp;
        return tail;
}

int main()
{
        struct node* tail = NULL;
        tail = createList(tail);
        printf("\nLinkedList before deletion: \n");
        print(tail);

        tail = delLast(tail);
        printf("\nLinkedList after deletion: \n");
        print(tail);
        return 0;
}
```

```
Enter the number of nodes of the linkedlist:
3
Enter the element 1: 1
Enter the element 2: 11
Enter the element 3: 111

LinkedList before deletion:
1 11 111

LinkedList after deletion:
1 11
```

➢Insert element in Sorted Singly Linkedlist

```c
#include<stdio.h>
#include<stdlib.h>

struct node{
        int data;
        struct node* link;
};

struct node* addToEmpty(int data)
{
        struct node* temp = malloc(sizeof(struct node));
        temp->data = data;
        temp->link = NULL;
        return temp;
}

struct node* addAtEnd(struct node* head, int data)
{
        struct node* newP = malloc(sizeof(struct node));
        newP->data = data;
        newP->link = NULL;

        struct node* temp = head;
        while(temp->link != NULL)
                temp = temp->link;
        temp->link = newP;
        return head;
}
```

```c
struct node* insert(struct node* head, int data)
{
        struct node* temp;
        struct node* newP = malloc(sizeof(struct node));
        newP->data = data;
        newP->link = NULL;

        int key = data;
        if(head == NULL || key < head->data)
        {
                newP->link = head;
                head = newP;
        }
        else
        {
                temp = head;
                while(temp->link != NULL && temp->link->data < key)
                        temp = temp->link;
                newP->link = temp->link;
                temp->link = newP;
        }
        return head;
}
```

Continued

```c
void print(struct node* head)
{
        struct node* temp = head;
        while(temp != NULL)
        {
                printf("%d  ", temp->data);
                temp = temp->link;
        }
        printf("\n");
}

int main(){
        int num;
        struct node* head = NULL;
        printf("number 10 added\n");
        head = addToEmpty(10);
        printf("number 20 added\n");
        addAtEnd(head, 20);
        printf("number 30 added\n");
        addAtEnd(head, 30);
        printf("number 40 added\n");
        addAtEnd(head, 40);

        printf("Enter the number: ");
        scanf("%d",&num);

        head = insert(head, num);
        printf("\nFinal Linkedlist: ");
        print(head);
        return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
number 10 added
number 20 added
number 30 added
number 40 added
Enter the number: 22

Final Linkedlist: 10  20  22  30  40
```

➢Doubly Linkedlist demonstration

```c
#include<stdio.h>
#include<stdlib.h>


struct node{
        struct node* prev;
        int data;
        struct node* next;
};

struct node* circularDoubly(int data)
{
        struct node* temp = malloc(sizeof(struct node));
        temp->prev = temp;
        temp->data = data;
        temp->next = temp;
        return temp;
}

int main(){

        int data=34;
        struct node* tail;
        tail = circularDoubly(data);

        printf("%d\n", tail->data);
        return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ gcc circular_doubly_linkedlist.c

┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
34
```

- In-order, pre-order and post-order (binary tree traversals) using recursion

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct node {
    int data;
    struct node *left;
    struct node *right;
} Node;

Node *create_node(int data) {
    Node *new_node = (Node*) malloc(sizeof(Node));
    new_node->data = data;
    new_node->left = NULL;
    new_node->right = NULL;
    return new_node;
}

void inorder_traversal(Node *root) {
    if (root == NULL) {
        return;
    }
    inorder_traversal(root->left);
    printf("%d ", root->data);
    inorder_traversal(root->right);
}

void preorder_traversal(Node *root) {
    if (root == NULL) {
        return;
    }
    printf("%d ", root->data);
    preorder_traversal(root->left);
    preorder_traversal(root->right);
}

void postorder_traversal(Node *root) {
    if (root == NULL) {
        return;
    }
    postorder_traversal(root->left);
    postorder_traversal(root->right);
    printf("%d ", root->data);
}

int main() {
    // Create a binary tree
    Node *root = create_node(1);
    root->left = create_node(2);
    root->right = create_node(3);
    root->left->left = create_node(4);
    root->left->right = create_node(5);
    root->right->left = create_node(6);
    root->right->right = create_node(7);

    // Perform traversals
    printf("Inorder Traversal: ");
    inorder_traversal(root);
    printf("\n");

    printf("Preorder Traversal: ");
    preorder_traversal(root);
    printf("\n");

    printf("Postorder Traversal: ");
    postorder_traversal(root);
    printf("\n");

    return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure/Run]
└─$ ./a.out
Inorder Traversal: 4 2 5 1 6 3 7
Preorder Traversal: 1 2 4 5 3 6 7
Postorder Traversal: 4 5 2 6 7 3 1
```

➢ Binary Search

```c
#include <stdio.h>

int binarySearch(int arr[], int left, int right, int target) {
    while (left <= right) {
        int mid = left + (right - left) / 2;

        // Check if target is present at middle position
        if (arr[mid] == target)
            return mid;

        // If target is greater, ignore left half
        if (arr[mid] < target)
            left = mid + 1;

        // If target is smaller, ignore right half
        else
            right = mid - 1;
    }

    // Target element was not found
    return -1;
}
```

```c
int main() {
    int arr[] = { 2, 4, 6, 8, 10, 12 };
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 8;

    int result = binarySearch(arr, 0, n - 1, target);
    if (result == -1)
        printf("Target element not found in the array");
    else
        printf("Target element found at index %d", result);

    return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
Target element found at index 3
```

➢Insertion sort

```c
#include <stdio.h>
// function to sort an array using insertion sort
void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
// Move elements of arr[0..i-1], that are greater than key,
// to one position ahead of their current position
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1; }
        arr[j + 1] = key;
    }
}
// function to print an array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
// main function to test the insertion sort
int main() {
    int arr[] = { 12, 11, 13, 5, 6 };
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original array:\n");
    printArray(arr, n);
    insertionSort(arr, n);
    printf("Sorted array:\n");
    printArray(arr, n);
    return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
Original array:
12 11 13 5 6
Sorted array:
5 6 11 12 13
```

➢ Bubble Sort

```c
#include <stdio.h>

void bubbleSort(int arr[], int n) {
    int i, j, temp;
    for (i = 0; i < n-1; i++) {
        // Last i elements are already sorted
        for (j = 0; j < n-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                // Swap arr[j] and arr[j+1]
                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr)/sizeof(arr[0]);
    printf("n= %d\n", n);
    bubbleSort(arr, n);
    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
n= 7
Sorted array: 11 12 22 25 34 64 90
```

➢Selection sort

```c
#include<stdio.h>
int main(){

int size = 7, a[size];
int i,j,min, temp;

printf("Enter 7 Numbers to perform selection sort:\n");
for(i=0; i<size ;i++)
        scanf("%d",&a[i]);

printf("\nBEFORE SELECTION SORT:\n");
        for(i=0 ; i<size ; i++)
                printf("%d\t", a[i]);

for(i=0; i<size-1; i++){
        min = i;
        for(j=i+1; j<size; j++){
                if(a[j] < a[i])
                {
                        temp = a[j];
                        a[j] = a[i];
                        a[i] = temp;
                }
        }
}

printf("\nAFTER SELECTION SORT:\n");
        for(i=0 ; i<size ; i++)
                printf("%d\t", a[i]);

}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
Enter 7 Numbers to perform selection sort:
123 2 11 346 785 99 239

BEFORE SELECTION SORT:
123     2       11      346     785     99      239
AFTER SELECTION SORT:
2       11      99      123     239     346     785
```

➢ Quick sort

```c
#include <stdio.h>
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i+1], &arr[high]);
    return i+1;
}

void quick_sort(int arr[], int low, int high) {
    if (low < high) {
        int pivot_index = partition(arr, low, high);
        quick_sort(arr, low, pivot_index - 1);
        quick_sort(arr, pivot_index + 1, high);
    }
}
```

```c
int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr)/sizeof(arr[0]);

    quick_sort(arr, 0, n-1);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
Sorted array: 1 5 7 8 9 10
```

➢Heap Sort

```c
#include <stdio.h>
// function to swap two elements
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// function to heapify a subtree rooted with
// node i which is an index in arr[]
void heapify(int arr[], int n, int i) {
    int largest = i;    // Initialize largest as root
    int left = 2*i + 1; // left child = 2*i + 1
    int right = 2*i + 2; // right child = 2*i + 2

    // If left child is larger than root
    if (left < n && arr[left] > arr[largest])
        largest = left;

    // If right child is larger than largest so far
    if (right < n && arr[right] > arr[largest])
        largest = right;

    // If largest is not root
    if (largest != i) {
        swap(&arr[i], &arr[largest]);

        // Recursively heapify the affected sub-tree
        heapify(arr, n, largest);
    }
}
```

CONTINUED

```c
// main function to do heap sort
void heapSort(int arr[], int n) {
    // Build heap (rearrange array)
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    // One by one extract an element from heap
    for (int i = n-1; i >= 0; i--) {
        // Move current root to end
        swap(&arr[0], &arr[i]);

        // call max heapify on the reduced heap
        heapify(arr, i, 0);
    }
}
// function to print an array
void printArray(int arr[], int n) {
    for (int i=0; i<n; ++i)
        printf("%d ", arr[i]);
    printf("\n");
}
// main function to test the heap sort
int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(arr)/sizeof(arr[0]);
    printf("Original array:\n");
    printArray(arr, n);
    heapSort(arr, n);
    printf("Sorted array:\n");
    printArray(arr, n);
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
Original array:
12 11 13 5 6 7
Sorted array:
5 6 7 11 12 13
```

- Merge sort

```c
#include <stdio.h>
#include <stdlib.h>

void merge(int arr[], int left[], int left_size, int right[], int right_size) {
    int i = 0, j = 0, k = 0;

    // merge the two sorted subarrays into the original array
    while (i < left_size && j < right_size) {
        if (left[i] <= right[j]) {
            arr[k] = left[i];
            i++;
        }
        else {
            arr[k] = right[j];
            j++;
        }
        k++;
    }

    // copy any remaining elements from the left subarray
    while (i < left_size) {
        arr[k] = left[i];
        i++;
        k++;
    }

    // copy any remaining elements from the right subarray
    while (j < right_size) {
        arr[k] = right[j];
        j++;
        k++;
    }
}

void merge_sort(int arr[], int size) {
    if (size <= 1) {
        return;  // base case: array of size 0 or 1 is already sorted
    }

    int mid = size / 2;
    int left[mid], right[size-mid];

    // copy the left half of the array into a new subarray
    for (int i = 0; i < mid; i++) {
        left[i] = arr[i];
    }

    // copy the right half of the array into a new subarray
```

Continued

```
    // copy the right half of the array into a new subarray
    for (int i = mid; i < size; i++) {
        right[i-mid] = arr[i];
    }

    // recursively sort the left and right subarrays
    merge_sort(left, mid);
    merge_sort(right, size-mid);

    // merge the sorted subarrays back into the original array
    merge(arr, left, mid, right, size-mid);
}

int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr)/sizeof(arr[0]);

    merge_sort(arr, n);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

```
┌──(kali㉿ft10101)-[~/Documents/Study/Datastructure]
└─$ ./a.out
Sorted array: 1 5 7 8 9 10
```